

STREAM NAVIGATOR — INSURANCE ENGINE AUDIT REPORT

StreamNavigator — Engine Audit
2026-09-20

An evaluation of `/insurance` — the page, the intake, the engine behind it, and whether a customer who pays \$79 gets more than \$79 back.

Conducted 2026-09-20 against production (`streamnavigator.ai/insurance`) and against `insurance.html`, `api/\_lib/navigator-engine.js`, `api/generate-paid-navigator.js`, `prices.config.json`, `navigator-shared.css` and `closing.html` at the repository's current `main`. **No paid model runs were spent** — the API budget is closed, so every finding below rests on the live page, one live intake probe that stopped before Stripe, and the engine's own source. `/closing` was read in full as the design and engineering reference the brief directs.

Executive Summary

Insurance Navigator sells a comparison it cannot reliably produce, and a verdict of "typical" it has no data to support.

The page promises: `"StreamNavigator compares the premium, limits, deductibles, and coverage against your old policy"` (`insurance.html:43`) and `"How much your premium changed, and whether that's typical"` (`insurance.html:84`). Behind that sits one thing: a single unstructured prompt in `PRODUCT_CONFIGS['insurance']` (`api/\_lib/navigator-engine.js:441`) handed to `claude-sonnet-5`, told to read whatever PDFs or images were uploaded and "compare... line by line" in one pass, with no extraction step, no structured fields, no diff, and no rate data anywhere in this codebase.

That matters here specifically because this repository already knows better. Six other Navigator products — Closing, Rental, Landlord, Home Savings, Subscriptions, Government Money — were each rebuilt in the last two weeks around the same finding: a bare model call reading documents and forming an opinion is not an audit, and it fails in specific, documented ways. The `HOME-SAVINGS-AUDIT.md` and `GOVERNMENT-MONEY-AUDIT.md` reports on this same codebase both landed on an identical rule — **never claim a price or a premium is "typical" or "market," because nobody holds a price table** — and wrote it into those products' prompts in capital letters. Insurance is the one flagship "stop overpaying" product that was never given that treatment. Its landing page still promises the exact ungrounded verdict this codebase's own audits found and banned everywhere else.

A second, structural gap sits under the headline promise: the intake requires only the renewal notice. The prior policy — the other half of "a comparison against your old policy" — is optional, flagged only as making the comparison "sharper." A customer who uploads just the renewal (the common case: most people do not have last year's declarations page filed away) pays \$79 for a product whose core mechanism, comparison, cannot run at all. The engine is instructed to notice this and say so (`"if only the renewal notice was given, work from that alone and say so explicitly"`), but nothing on the page prepares the customer for that outcome before they pay.

What works: the price is wired correctly (page shows \$79, Stripe link and `prices.config.json` agree), client-side validation is real and inline (verified live — email and file requirements block submission with a visible error, not a toast that disappears), the pricing structure (one-time per renewal) is the right shape for a once-a-year decision, and the product is honest about what it will not do — the FAQ states plainly that it will not shop for a new policy on the customer's behalf. That restraint is worth preserving; it is the same discipline that kept `/home-savings` from promising a market comparison it can't make.

Biggest customer-value problem: the product's central deliverable — a verdict on whether a premium

increase is "typical," and whether the renewal is "worth shopping" — is not backed by anything that could make that verdict reliable. It is one model's read of two documents it may or may not both have.

Biggest product opportunity: everything needed to fix this is arithmetic, not a market database. A renewal notice and a prior policy are two documents with a fixed, extractable shape (premium, per-coverage limits, deductibles, named exclusions). Diffing those two structured extractions deterministically — the same move made for Closing Disclosures, rent rolls, and utility bills — turns "whether that's typical" (unverifiable) into "your dwelling limit dropped \$40,000 while your premium rose \$310" (a fact, reproducible, and exactly what a customer needs to bring to their agent).

Is the current price justified? No, not for what ships today — a single ungrounded LLM read is priced the same as this codebase's actual deterministic audits. **Recommended price: \$79/renewal, conditional** — see Pricing Audit below for what has to ship first, matching the standard this codebase already applied to Landlord (\$149) and Government Money (\$39): the higher price was for the fixed product, not the one currently live.

Promise vs. Delivery

Promise	What the Product Actually Does	Gap	Severity	Recommended Fix
"StreamNavigator compares the premium, limits, deductibles, and coverage against your old policy" (`insurance.html:43`)	One model call reads whatever files were attached and writes a comparison in prose. The prior policy is optional; when absent, there is nothing to compare against.	No deterministic extraction or diff exists; the "comparison" is conditional on a document the intake does not require.	Critical	Build a deterministic extract !' diff engine (see Decision Engine Audit) matching the closing/rental/landlord pattern. Until it ships, make the prior policy required, or clearly discount/ relabel the product when it's absent.
"How much your premium changed, and whether that's typical" (`insurance.html:84`)	The model is told to "give a clear verdict on whether the renewal looks fair/typical" (`navigator-engine.js:446`) from general training knowledge — no rate table, no regional data, no carrier filings anywhere in the codebase.	This is precisely the claim this codebase's own audits (`HOME-SAVINGS-AUDIT.md`, `GOVERNMENT-MONEY-AUDIT.md`) identified as unverifiable and banned in every other product's prompt. Insurance's prompt still asks for it.	Critical	Remove "typical" as a claim. Keep the arithmetic (premium changed by \$X / Y%) — that's real, in-document math — and drop the market judgment, replacing it with a concrete flag ("a change above N% is worth a direct question to your insurer") rather than an opinion on normalcy.

Promise	What the Product Actually Does	Gap	Severity	Recommended Fix
"New exclusions or coverage gaps introduced in this renewal" (`insurance.html:87`)	Read in the same single unstructured pass as everything else; no per-document exclusion list is extracted or diffed.	This codebase has already observed this exact failure mode on Rental: an identical single-pass read missed a real \$118/mo charge on one property and volunteered a charge that didn't exist on another (`navigator-engine.js:266-270`, rental-engine commentary). Nothing here checks a list.	High	Extract exclusions/endorsements into a structured list per document (a fixed, enumerable field on nearly every policy form) and diff the lists, rather than asking the model to notice differences by reading.
"Specific questions to ask your insurer"	Generated by the model from the `sections` field of the shared report schema. Plausible and reasonably low-risk regardless of extraction quality.	None material — this is the one deliverable a single well-prompted model call can do adequately without a deterministic backbone.	Low	No change needed; keep as the one part of the report the model authors directly.
"Whether it's worth shopping this renewal elsewhere"	A verdict from the same ungrounded pass, with no risk-of-coverage-gap or underinsurance guardrail attached.	The brief's own Section 14 standard — minimize cost while preserving protection, never optimize for premium alone — is not encoded anywhere in the prompt.	High	Add an explicit risk-safeguard block to the prompt (see Risk Protection Audit) and a standing caution in the output about not lapsing current coverage while shopping.
"\$79/renewal, one flat price" (`insurance.html:126`)	Matches `prices.config.json:44-48` (7900 cents) and the live Stripe link exactly. Verified — no drift.	None.	—	No fix needed.
Footer: "AI-generated analysis... not licensed insurance advice" (`insurance.html:242`)	Present, matches the pattern used on every other Navigator page.	None.	—	No fix needed.

Customer Journey Audit

1. Landing page

Works: The hero states the problem ("renewals auto-approve if you do nothing"), the mechanism (upload, compare against your old policy), and the price is visible by scrolling one section. A rational visitor understands what this is within about 15 seconds. **Doesn't:** There is no example of the output anywhere above the fold or below it — no sample "before !" after" line, no mock finding. `closing`'s hero carries a small live-looking table showing an actual finding ("Lender fee... \$300.00 — possible overcharge") right next to the headline; a visitor sees the mechanism working before they scroll. Insurance has nothing equivalent, so the promise is asserted, not demonstrated. **Fix:** Add one concrete, clearly-labeled example finding near the hero (e.g., "Dwelling coverage: \$340,000 !' \$310,000. Premium: \$1,140 !' \$1,410.") — real arithmetic the product can actually produce once the diff engine below ships, not a mocked-up number pretending to be live.

2. Input process

Works: Category chips (Auto/Home/Renters/Umbrella/Other), an optional free-text "what's changed" box, file upload (PDF/JPG/PNG, 50MB/file), and email. Verified live: submitting with no email produces an inline, persistent error ("Enter a valid email so we can reach you.") rather than a toast that vanishes — this matches the fix already shipped site-wide for the "reload the page, attach files, click, and see nothing" defect class. The upload requirement is enforced client-side (`insurance.html:285`) and server-side (`requiresFiles: true`, `navigator-engine.js:443`). **Doesn't:** Only one file is actually required. The prior policy — without which the headline feature cannot execute — is optional and undersold ("makes the comparison significantly more precise," `insurance.html:187`, rather than "without this we cannot compare anything"). A customer with no scanned copy of last year's policy will submit successfully, pay \$79, and get a materially different, weaker product than the one described in the hero. **Fix:** Either require both documents (matching how Closing requires the Closing Disclosure and treats the Loan Estimate as materially improving but not blocking the *audit*, which still runs), or make the intake honest about the degraded outcome before checkout — a visible note next to the optional upload: "Without your prior policy, you'll get a read of this renewal alone, not a comparison."

3. Analysis

Covered in depth in Decision Engine Audit below. In short: one model call, no extraction, no diff, no rate data, asked to render a verdict this codebase's own prior audits determined cannot be made honestly.

4. Results / Recommendation

The report renders through the same generic template every non-bespoke Navigator product uses (`navigator-status.html`: headline, key numbers, labeled sections, an honest "what I couldn't verify" list). The shape is fine — it is the same shape Closing and Rental use to deliver genuinely deterministic findings. The problem is only that, for Insurance, nothing upstream constrains what goes into it. `key\_numbers` has no "kind" discipline (confirmed vs. at-risk vs. unpriced) the way Home Savings and Subscriptions enforce — nothing stops the model from putting an invented dollar savings estimate in large type at the top of the

report.

5. Next action

FAQ: *"Will you shop for a new policy for me? No — you get a clear read on your current renewal plus guidance on what to shop for, but you make contact with insurers yourself."* This is honest and correctly scoped — it does not overpromise a live-shopping capability the product doesn't have. Keep it.

6. Payment / pricing

\$79 one-time per renewal, correctly wired end to end (verified against `prices.config.json` and the live Stripe link). One-time-per-renewal is the right pricing *shape* — insurance renewals are annual events, and a subscription would bill the customer eleven months they have nothing to review. No structural change needed here; see Pricing Audit for whether the *amount* is earned by the current build.

Insurance Decision Engine Audit

There is, at present, no decision engine. `PRODUCT_CONFIGS['insurance']` (`api/_lib/navigator-engine.js:441-447`) is 130 words of task instruction handed straight to the model alongside the raw uploaded files — the same shape used for Property Tax and Home Maintenance, the two other Navigator products that have not yet been rebuilt around a deterministic core, and unlike the six products (Closing, Rental, Landlord, Home Savings, Subscriptions, Government Money) whose `PRODUCT_CONFIGS` entries explicitly say *"the findings in this report are produced by [engine file], not by the model. The model's job is to present them. It must not originate a number, a threshold, or a recommendation."* Insurance has no such engine to defer to, so its prompt instructs the opposite: the model **is** the analyst.

Strong decision rules: none exist to evaluate — there are no rules, only a task description.

Weak decision rules:

- "Give a clear verdict on whether the renewal looks fair/typical" — asks for a market judgment with no market data behind it.
- "Compare... line by line" — an instruction to a single-pass reader,

not an actual line-by-line mechanism. Nothing enumerates the lines to check, so completeness depends entirely on what the model happens to notice in one read of two documents it may not both have.

Missing decision rules (all directly answerable from the brief's own Section 5, which the current prompt does not encode at all):

- No enumerated list of **likely-justified** increase reasons (replacement

cost/dwelling-value update, claims history, general rate filings, deductible or limit changes the customer themselves requested) versus **worth-challenging** reasons (a rating factor that looks wrong, a discount that disappeared, coverage that no longer matches the customer's situation, a renewal increase disconnected from any coverage change). Every other rebuilt product in this codebase (Landlord's jurisdiction gates, Government Money's four eligibility rules, Subscriptions' 12 named rules) encodes exactly this kind of enumerated rule set. Insurance has none.

- No rule for the size of a percentage change that warrants flagging versus

one that doesn't — currently left entirely to the model's judgment call, applied inconsistently by definition.

- No handling of the "only the renewal notice was provided" case beyond

"say so" — there is no instruction to scope the verdict down proportionally (e.g., refuse to render a shopping recommendation at all without a comparison baseline, rather than a softened version of one).

Incorrect / dangerous assumptions:

- Treating "typical" as something a general-purpose language model can

determine from training data is the same assumption this codebase's own ``GOVERNMENT-MONEY-AUDIT.md`` found and rejected for program dollar amounts, and ``HOME-SAVINGS-AUDIT.md`` rejected for bill pricing. It is no more reliable here. A premium that rose because of a 2025-2026 regional reinsurance repricing (real, current, and not something a model's training data can be trusted to have current numbers on) can be told to a customer as "not typical — worth challenging" with total confidence and no basis.

Opportunities for better analysis (what a real engine would do, and what none of it requires a live rate lookup to build):

1. Extract both documents into a fixed schema: named coverage lines, limits,

deductibles, premium, endorsements/exclusions, effective dates — the same move ``closing-extract.js``, ``rental-extract.js``, and ``home-savings-extract.js`` already make for their respective document types.

1. Diff the two extractions deterministically: which lines changed, by how

much, in which direction.

1. Apply the justified/worth-challenging categorization above as a rule

set, not a model opinion — flag "premium rose but no coverage line changed" as worth-challenging; flag "dwelling limit rose with the premium" as likely-justified, by rule, every time, the same way.

1. Hand the model only the diff and the categorized flags to write up — the

same "the model presents, it does not decide" boundary already enforced everywhere else in this codebase.

Savings Engine Audit

How savings are identified: They aren't, in any disciplined sense. The shared ``key_numbers`` field is open to the model with no "kind" label the way Home Savings and Subscriptions require ("confirmed" vs. "at_risk" vs. "unpriced" — ``navigator-engine.js:220-224``). Nothing stops an Insurance report from putting an invented annual-savings figure in large type at the top of the page.

How savings are estimated: There is no savings estimation logic at all — switching-carrier savings cannot be known without a competing quote, which this product does not obtain (correctly — see FAQ). The only number this product can *honestly* produce is the dollar and percentage change between the two documents it was given, which is arithmetic, not an estimate, and currently is not guaranteed to appear because nothing forces the model to compute and check it.

Whether savings are credible: Not as currently built. The product's only credible number — the in-document premium delta — is not distinguished from its incredible one — a market/typical judgment — anywhere in the schema or the prompt.

Whether savings are communicated clearly: Structurally yes (the shared report template is clean and has shipped successfully on six other products); substantively no, because what fills that template for Insurance is not yet disciplined the way it is for those six.

Whether savings justify the customer's cost: See Economic Value below.

Risk Protection Audit

The brief's Section 14 standard — minimize unnecessary cost while preserving appropriate protection, never optimize for the lowest premium alone — is not encoded anywhere in ``PRODUCT_CONFIGS['insurance']``. Compare this to how carefully the same discipline is written into other products in this exact file: Subscriptions' prompt refuses to let the model recommend cancelling a shared plan or something holding the customer's data "because the engine does not know what they use each for and neither do you" (``navigator-engine.js:349``); Landlord's prompt won't let severities be upgraded past what the deterministic engine decided. Insurance has no equivalent guardrail against recommending a customer shop away from or reduce coverage.

Two concrete gaps:

1. **No underinsurance guardrail.** Nothing instructs the model to weigh

whether a lower-premium alternative it points the customer toward would also mean lower limits, a higher deductible, or a dropped endorsement — the exact false economy Section 14 exists to prevent.

1. **No coverage-lapse warning.** The CTA copy ("Renewals auto-approve if

you do nothing... spend two minutes checking first") correctly creates urgency to review before the renewal date, but nothing in the product — page or prompt — tells the customer not to let their *current* policy lapse while they shop a flagged renewal. A gap in coverage while switching carriers is a real, common, entirely avoidable risk this product currently has no opinion on.

Fix: Add an explicit risk-safeguard clause to the prompt (mirroring the Subscriptions/Landlord pattern) and a standing item in the output's closing section: confirm replacement coverage is bound before cancelling anything, and never let a premium comparison alone drive a recommendation to reduce limits or deductible protection.

UX Audit

Design: Clean, legible, consistent with the other ten Navigator product pages that share ``navigator-shared.css`` — bold black-outline cards, hard drop shadows, a violet/mint/pink gradient palette, Inter body text and Sora headlines.

The comparison the brief asks for is complicated by one fact: ``/closing``, named as the design reference, does **not** use ``navigator-shared.css`` at all. It carries its own, entirely separate stylesheet

(`closing.html:11-40`+): Newsreader serif headlines, IBM Plex Sans body text, IBM Plex Mono for figures, a muted paper/ink/flag-red palette, and a plain 1px-rule layout with no gradients or hard shadows — a deliberately quieter, more editorial register built for a product about auditing a legal document. Matching Insurance to Closing pixel-for-pixel would pull it **out** of step with the other ten Navigator pages it currently matches, not into step with them.

Specific, verifiable inconsistencies between `insurance` and `closing`:

Element	`closing`	`insurance`
Headline font	Newsreader (serif)	Sora (bold sans)
Body font	IBM Plex Sans	Inter
Figures/numbers	IBM Plex Mono, tabular	No monospace treatment
Color language	Muted paper (#FBFAF7), ink-blue (#1B2A3A), a single flag-red for findings	Violet/mint/pink gradient, hard black shadows
Card treatment	1px rule borders, flat	Hard 6px offset shadow, heavier visual weight
Proof element	Live-looking finding table in the hero	None
Overall register	Serious, document-audit, low-saturation	Playful, bright, SaaS-marketing

Judgment call, stated plainly per the brief's instructions: Closing is a document-grounded, annual-premium decision with recommended fix is **not** a wholesale re-skin to match Insurance's site further — now three visual systems instead of two: the hero's saturated gradient text treatment for the price and any dollar figures element. This brings Insurance closer to Closing's serious products belong to. If ntari wants full visual convergence, a larger decision — re-platforming Closing onto the shared

Mobile: Verified live at 375×812. Renders cleanly, text is usable. No CTA is visible without scrolling past the hero. Not a page-specific defect.

Friction: Input process is short (5 fields, one required), excessive disclaimers, no vague claims found in the closing above.

Missing example: No sample finding, no mocked comparison, nothing that lets a visitor see what a \$79 report actually looks like before paying for one. This is the single highest-leverage UX fix, because it is also the fix that would force the product to prove — in its own marketing — that it can produce the concrete, arithmetic-grounded finding this report recommends building.

Pricing Audit

Current Price: \$79/renewal, one-time (verified: ``insurance.html:126`, `prices.config.json:44-48``, live Stripe link ``3cl9ATaosfqQ88k7ZSabK0b`` all agree).

Recommended Price: \$79/renewal — **conditional**, not automatic.

Recommended Pricing Model: Unchanged. One-time per renewal is the correct structure: the customer's need recurs on their own policy's renewal cycle (typically annual, sometimes six-monthly for auto), not on a subscription clock. A monthly or annual subscription would bill for reviews the customer doesn't need most months. Keep the flat per-renewal fee.

Reasoning:

This mirrors the standard this codebase has already applied twice — Landlord stayed at \$149 and Government Money returned to \$39 same-day, in both cases because the audited number was **for the fixed product**, with named conditions attached, not a reason to cut the live price of the broken one. The same logic applies here, in the same direction:

Economics of the fixed product. A renewal-vs-prior-policy comparison, done properly (structured extraction of two documents, a deterministic diff, a categorized justified/worth-challenging read, drafted questions for the insurer) is comparable in scope to Closing Disclosure Audit (\$59, twenty-eight named checks) or Landlord (\$149, jurisdiction-matched compliance). Auto and home insurance premiums commonly move 5-20%+ a year in the current market; a customer who uses a genuinely reliable "worth challenging" call to negotiate or shop even a modest correction on an \$1,800/year home policy or \$2,000/year auto policy is realistically looking at \$100-\$400 in first-year savings, recurring if they act on it. Against \$79, that is a defensible ROI for a meaningful share of customers — not all, since many renewals genuinely are fair, in which case the honest deliverable is confirmation, not savings (the same "clean bill of health is still the product" principle this codebase already applies to Closing and Rental).

Economics of what actually ships today. A single, ungrounded model read of whatever was uploaded — sometimes just the renewal, with no baseline to compare against at all — is a materially thinner product than that. Priced at \$79, it sits above Property Tax's ungrounded single-pass product only because Property Tax is \$99 for a different reason (no document requirement at all, "we can often work from the address alone"); it sits well above Home Maintenance's \$59 for the same architecture. There is no principled basis in this codebase for Insurance's **current** build to be priced higher than a comparable ungrounded product, and every reason the **promise** on the page (comparison, "typical," exclusion detection) requires the deterministic engine described above before \$79 is earned.

Conditions for \$79 to stand, matching the standard set for Landlord and Government Money:

1. A deterministic extraction + diff engine ships (Decision Engine Audit,

Critical fixes below) — the comparison the page sells must actually run as a mechanism, not a single read.

1. The "typical" claim is either removed from the page and the prompt, or replaced with something the product can actually back up.

1. The prior policy is required, or the price/promise is visibly adjusted

when it's absent (a customer paying full price for a renewal-only read is being sold something the page does not describe).

If these are not shipped, the honest price for what exists today is in the \$39-49 range — in line with this codebase's other still-unrebuilt, document-optional, single-pass products — until the engine work above lands.

Required Changes

Critical — Must Fix

1. **Build a deterministic extraction + diff engine for Insurance**

(`insurance-extract.js` / `insurance-audit.js`, matching the shape of `closing-extract.js` + `closing-audit.js`): structured fields for premium, per-coverage limits, deductibles, and named exclusions/endorsements from both documents, diffed line by line. This is the mechanism the page already claims exists.

1. **Remove or ground the "typical" claim.** Either drop "whether that's

typical" from the page and the prompt, or attach it to something real (a stated percentage threshold, not a market judgment) — this is the exact claim this codebase's own prior audits found and banned elsewhere.

1. ****Require the prior policy, or make its absence visibly change the**

product before checkout.** The core promise cannot execute without it; currently a customer can pay full price and never find that out until after they've paid.

High Priority

1. **Encode the justified-vs-worth-challenging rule set** (Section 5 of this brief) as explicit rules in the engine, not left to model judgment.

1. **Add the missing risk-protection guardrail** — no coverage-reduction

recommendation without weighing the tradeoff, plus a standing coverage-lapse warning in every report's closing section.

1. **Add a "kind" discipline to any dollar figure** the report states

(in-document arithmetic vs. anything else), matching Home Savings and Subscriptions, so an invented savings estimate cannot appear in large type at the top of a report.

Medium Priority

1. **Add a concrete example finding near the hero** so a visitor can see

the product's actual mechanism before paying — the single highest- leverage UX fix, and one that forces the marketing claim to match a real capability once Critical #1 ships.

1. **Lighter visual alignment toward Closing's seriousness** within the

existing shared design system (ink-toned headline instead of the gradient treatment, monospace numerals for price and report figures) — not a full re-skin, which would fragment the site further.

Low Priority

1. Tighten the FAQ's first answer ("Auto, home, renters, and umbrella

policies most commonly") to state plainly which categories the engine is actually tuned for once the rule set in #4 exists, rather than "we'll do our best" for the rest.

Final Customer-Value Test

If I were a rational customer paying \$79, would I reasonably expect this product to save me more money than it costs while helping me avoid a materially worse insurance decision?

Customer Value Verdict: Not reliably, as currently built. The product's one differentiated claim — a comparison against the prior policy, with a verdict on whether a premium change is typical — either cannot run (no prior policy required) or rests on a model's ungrounded read where this codebase's own prior audits have already shown that exact failure mode costs customers real money elsewhere (the missed PMI charge on Rental). What the product *can* currently deliver reliably — a few plausible questions to ask an insurer — is not, by itself, worth \$79. The FAQ's honesty about not shopping on the customer's behalf, and the correctly-wired \$79 one-time price, are real positives that should be kept, not the reasons the current price is earned.

Recommended Product: A renewal notice and prior policy are extracted into a structured schema (premium, per-line limits, deductibles, exclusions), diffed deterministically, and each change categorized by rule as likely-justified or worth-challenging. The model's role narrows to writing up findings it did not originate — the same boundary already enforced on six other products in this codebase. The output leads with the in-document arithmetic (what changed, by how much, in dollars), never claims a "typical" market judgment, names specific questions grounded in the actual diffed changes, and carries an explicit risk-protection line against recommending coverage reduction or letting current coverage lapse while shopping.

Recommended Price: \$79/renewal, one-time — conditional on the three Critical fixes above shipping. Absent them, \$39-49/renewal is what the current, ungrounded single-pass build actually earns.

Highest-Impact Fix: Build the deterministic extraction + diff engine. Every other finding in this report — the ungrounded "typical" claim, the optional prior policy undermining the core promise, the missing risk guardrail, the missing example on the landing page — either is caused by its absence or becomes straightforward to fix once it exists.